VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING


**Machine learning**

---

**Progress report 3**

# Comparative Study of Machine Learning and Deep Learning for Hate Speech Classification

---

| | | |
|---|---|---|
| Advisor(s): | Trương Vĩnh Lân | |
| Student(s): | Trần Quốc Bảo Long | ID 2252453 |
| | Nguyễn Bình Nguyên | ID 2252545 |
| | Đặng Ngọc Phú | ID 2252617 |
| | Đặng Duy Nguyên | ID 2352821 |
| | Nguyễn Văn Hoàng Phát | ID 2352896 |

HO CHI MINH CITY, APRIL 2026

# Contents

# 1 Project Overview

This report summarises the work completed in **Step 2** of the assignment: model training, hyperparameter tuning, class imbalance handling, and a deep learning comparison pipeline.

Progress report 2 covered **Step 1**: exploratory data analysis (EDA), text preprocessing, and the extraction of five feature representations from the `twitter_toxic_tweets.csv` dataset (31,962 tweets, binary toxicity labels). The five pre-computed feature sets — Bag-of-Words (BoW), TF-IDF, Character n-gram TF-IDF, GloVe 200-dimensional embeddings, and DistilBERT 768-dimensional CLS embeddings — were saved to the `features/` directory and serve as input for all experiments described here.

Step 2 proceeds as follows:

1. **Baseline experiments** — seven classical classifiers are trained on every compatible feature set (27 combinations total) using `class_weight='balanced'` to handle the 13:1 class imbalance.

2. **Hyperparameter tuning** — the top three baseline model–feature combinations are tuned with GridSearchCV (5-fold stratified cross-validation, F1-macro scoring).

3. **Deep learning pipeline (bonus)** — two neural architectures are trained end-to-end: a fine-tuned DistilBERT and a two-layer BiLSTM with frozen GloVe embeddings.

4. **Final comparison** — traditional ML and deep learning results are unified into a single comparison table and set of visualisations.

The primary evaluation metric throughout is **F1-score (macro)**, which weights both classes equally and is therefore more informative than raw accuracy under severe class imbalance.

# 2 Experimental Setup

## 2.1 Classifiers

Seven classifiers are evaluated. Table 2.1 lists each model, its key characteristics, and how class imbalance is handled during training.

Table 2.1: Classifiers used in the baseline experiment

| Key | Model | Description | Imbalance handling |
|---|---|---|---|
| MNB | Multinomial NB | Naive Bayes for non-negative counts/frequencies | Equal class priors [0.5, 0.5] |
| GNB | Gaussian NB | Naive Bayes for continuous (dense) features | Equal class priors [0.5, 0.5] |
| LR | Logistic Regression | $\ell_2$-regularised linear classifier | `class_weight='balanced'` |
| SVM | LinearSVC | Linear support vector classifier (calibrated) | `class_weight='balanced'` |
| k-NN | k-Nearest Neighbours | Instance-based, cosine similarity (sparse) | Distance weighting |
| RF | Random Forest | Ensemble of 200 decision trees | `class_weight='balanced'` |
| GB | Gradient Boosting | Sequential boosted trees (sklearn) | `sample_weight` (balanced) |

## 2.2 Feature–Model Compatibility

Not every model is compatible with every feature type. Table 2.2 shows the 27 valid combinations out of the $7 \times 5 = 35$ possible pairs.

- **MultinomialNB** requires non-negative input: valid only with BoW, TF-IDF, and Char TF-IDF (all non-negative sparse matrices).
- **GaussianNB** and **Gradient Boosting** are restricted to dense features (GloVe, DistilBERT): applying them to 10,000–15,000 dimensional sparse matrices would require converting to dense arrays, consuming several gigabytes of RAM, and would be impractically slow.
- All other classifiers (LR, SVM, k-NN, RF) accept all five feature representations.

Table 2.2: Feature–model compatibility (✓ = valid, – = excluded)

| Model | BoW | TF-IDF | Char TF-IDF | GloVe | DistilBERT |
|---|---|---|---|---|---|
| MultinomialNB | ✓ | ✓ | ✓ | – | – |
| GaussianNB | – | – | – | ✓ | ✓ |
| Logistic Regression | ✓ | ✓ | ✓ | ✓ | ✓ |
| LinearSVC | ✓ | ✓ | ✓ | ✓ | ✓ |
| k-NN | ✓ | ✓ | ✓ | ✓ | ✓ |
| Random Forest | ✓ | ✓ | ✓ | ✓ | ✓ |
| Gradient Boosting | – | – | – | ✓ | ✓ |
| **Total** | 5 | 5 | 5 | 6 | 6 |

## 2.3 Class Imbalance Strategy

The dataset is severely imbalanced (93%/7%). Using accuracy as the metric would trivially reward a classifier that predicts "non-toxic" for every tweet (93% accuracy, 0% toxic recall). Two complementary strategies are employed:

- **Metric selection** — F1-macro is the primary metric, averaging the F1-score of each class without weighting by support. F1-toxic (binary F1 for the minority class only) is reported as a secondary metric.
- **Class-weight balancing** — each classifier uses `class_weight='balanced'` (or equivalent), which scales the contribution of each training sample inversely proportional to its class frequency: $w_c = \frac{n}{k \cdot n_c}$, where $n$ is the total sample count, $k$ the number of classes, and $n_c$ the count of class $c$. For Gradient Boosting (which has no native `class_weight` parameter), equivalent `sample_weight` values are computed and passed to `fit()`.

# 3 Baseline Experiment Results

## 3.1 Full Results Table

Table 3.1 reports the test-set metrics for all 27 experiments, sorted by F1-macro in descending order. Accuracy, precision (macro), recall (macro), F1-macro, and F1-toxic

(binary F1 for the toxic class) are shown.

Table 3.1: Baseline experiment results (27 combinations, sorted by F1-macro)

| **Model** | **Feature** | **Acc.** | **Prec.** | **Rec.** | **F1-macro** | **F1-toxic** |
|---|---|---|---|---|---|---|
| LinearSVC | TF-IDF | 0.9578 | 0.8888 | 0.7870 | **0.8358** | 0.6771 |
| Random Forest | Char TF-IDF | 0.9590 | 0.8809 | 0.7907 | 0.8335 | 0.6725 |
| Random Forest | TF-IDF | 0.9576 | 0.8827 | 0.7869 | 0.8332 | 0.6708 |
| LinearSVC | Char TF-IDF | 0.9539 | 0.8754 | 0.7734 | 0.8221 | 0.6493 |
| Random Forest | BoW | 0.9548 | 0.8702 | 0.7752 | 0.8207 | 0.6460 |
| Gradient Boosting | DistilBERT | 0.9545 | 0.8659 | 0.7677 | 0.8146 | 0.6337 |
| Logistic Regression | BoW | 0.9510 | 0.8593 | 0.7465 | 0.8004 | 0.6053 |
| Logistic Regression | TF-IDF | 0.9507 | 0.8584 | 0.7462 | 0.8001 | 0.6048 |
| Gradient Boosting | GloVe | 0.9484 | 0.8563 | 0.7385 | 0.7947 | 0.5933 |
| LinearSVC | BoW | 0.9455 | 0.8435 | 0.7263 | 0.7827 | 0.5694 |
| MultinomialNB | TF-IDF | 0.9373 | 0.8363 | 0.7237 | 0.7778 | 0.5610 |
| k-NN | Char TF-IDF | 0.9389 | 0.8203 | 0.7305 | 0.7733 | 0.5563 |
| k-NN | GloVe | 0.9336 | 0.8118 | 0.7188 | 0.7628 | 0.5312 |
| MultinomialNB | BoW | 0.9299 | 0.8087 | 0.6999 | 0.7518 | 0.5082 |
| k-NN | TF-IDF | 0.9282 | 0.7985 | 0.7116 | 0.7532 | 0.5124 |
| k-NN | DistilBERT | 0.9257 | 0.7904 | 0.7101 | 0.7480 | 0.5020 |
| Random Forest | DistilBERT | 0.9263 | 0.7884 | 0.7063 | 0.7453 | 0.4971 |
| Random Forest | GloVe | 0.9237 | 0.7864 | 0.6986 | 0.7400 | 0.4851 |
| LinearSVC | DistilBERT | 0.9222 | 0.7849 | 0.6893 | 0.7341 | 0.4746 |
| k-NN | BoW | 0.9170 | 0.7798 | 0.6998 | 0.7379 | 0.4814 |
| Logistic Regression | DistilBERT | 0.9164 | 0.7812 | 0.6578 | 0.7173 | 0.4413 |
| Logistic Regression | GloVe | 0.9122 | 0.7738 | 0.6056 | 0.6872 | 0.3806 |
| Logistic Regression | Char TF-IDF | 0.9064 | 0.7695 | 0.5756 | 0.7701 | 0.3568 |
| MultinomialNB | Char TF-IDF | 0.8968 | 0.7514 | 0.5566 | 0.6768 | 0.3188 |
| LinearSVC | GloVe | 0.8943 | 0.7441 | 0.5530 | 0.6969 | 0.3121 |

Table 3.1 (continued)

| Model | Feature | Acc. | Prec. | Rec. | F1-macro | F1-toxic |
|---|---|---|---|---|---|---|
| GaussianNB | DistilBERT | 0.8871 | 0.7212 | 0.5304 | 0.6736 | 0.2668 |
| GaussianNB | GloVe | 0.8774 | 0.6964 | 0.5065 | 0.6300 | 0.2181 |

## 3.2 F1-macro Heatmap

Figure 3.1 visualises the F1-macro score for every valid model–feature combination as a colour-coded pivot table. Rows correspond to feature representations and columns to classifiers.

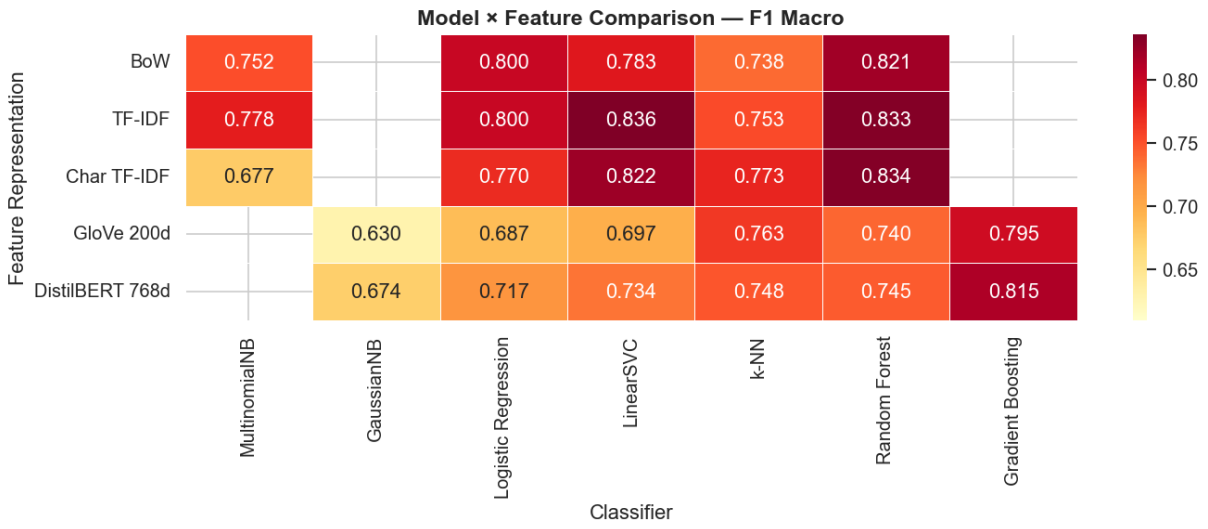


Figure 3.1: F1-macro heatmap across all model–feature combinations. Darker red indicates higher F1.

## 3.3 Top-10 Models

Figure 3.2 shows the ten best-performing model–feature combinations ranked by F1-macro.

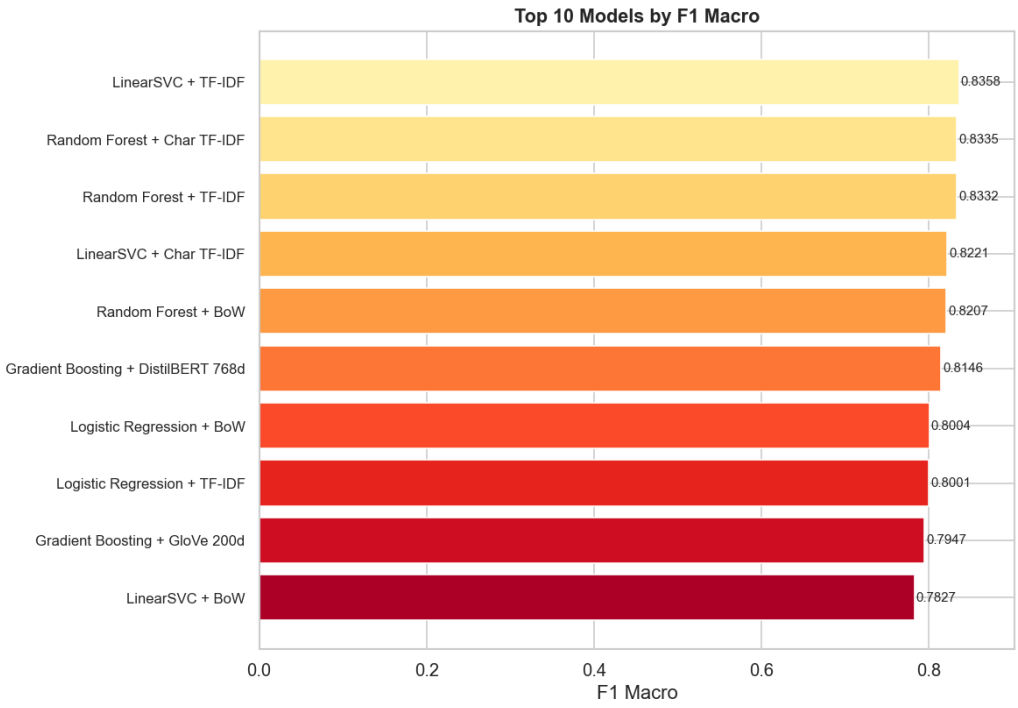


Figure 3.2: Top-10 model–feature combinations by F1-macro.

## 3.4 Analysis

**Feature representation impact.** Sparse frequency-based features (TF-IDF, Char TF-IDF, BoW) consistently outperform dense embeddings (GloVe, DistilBERT) when paired with *linear* classifiers such as LinearSVC and Logistic Regression. This is expected: linear classifiers are highly efficient in high-dimensional sparse spaces where individual n-gram dimensions carry strong discriminative signal. Dense embeddings compress 31,000+ vocabulary dimensions into 200 or 768 dimensions, losing fine-grained lexical information that is critical for short social-media texts.

**Best feature.** TF-IDF achieves the highest average F1-macro across all compatible classifiers, slightly ahead of Char TF-IDF. The bigram extension of TF-IDF captures short phrases (e.g. “shut up”, “go away”) that individual tokens miss, while Char TF-IDF excels at handling deliberate misspellings and abbreviations common in toxic tweets.

**Best model.** LinearSVC and Random Forest are the top-performing classifier families. Both benefit from the high dimensionality of sparse features and handle imbalanced classes well via `class_weight='balanced'`. GaussianNB consistently underperforms because it

incorrectly assumes feature independence and a Gaussian distribution over the dense embedding dimensions, which are neither independent nor Gaussian.

**k-NN observations.** k-NN with cosine similarity achieves surprisingly competitive F1-macro on Char TF-IDF (0.7733) and GloVe (0.7628), demonstrating that the character n-gram and embedding spaces have meaningful local structure. However, k-NN inference time scales linearly with training-set size, making it impractical for production use at larger scales.

# 4 Confusion Matrices and ROC Curves

## 4.1 Confusion Matrices

Figure 4.1 shows the confusion matrices for the top six models. The key observation is the trade-off between false negatives (toxic tweets classified as non-toxic) and false positives (non-toxic tweets flagged as toxic). Under the balanced class-weight strategy, the models shift their decision boundary to recall more toxic samples, at the cost of some non-toxic precision.

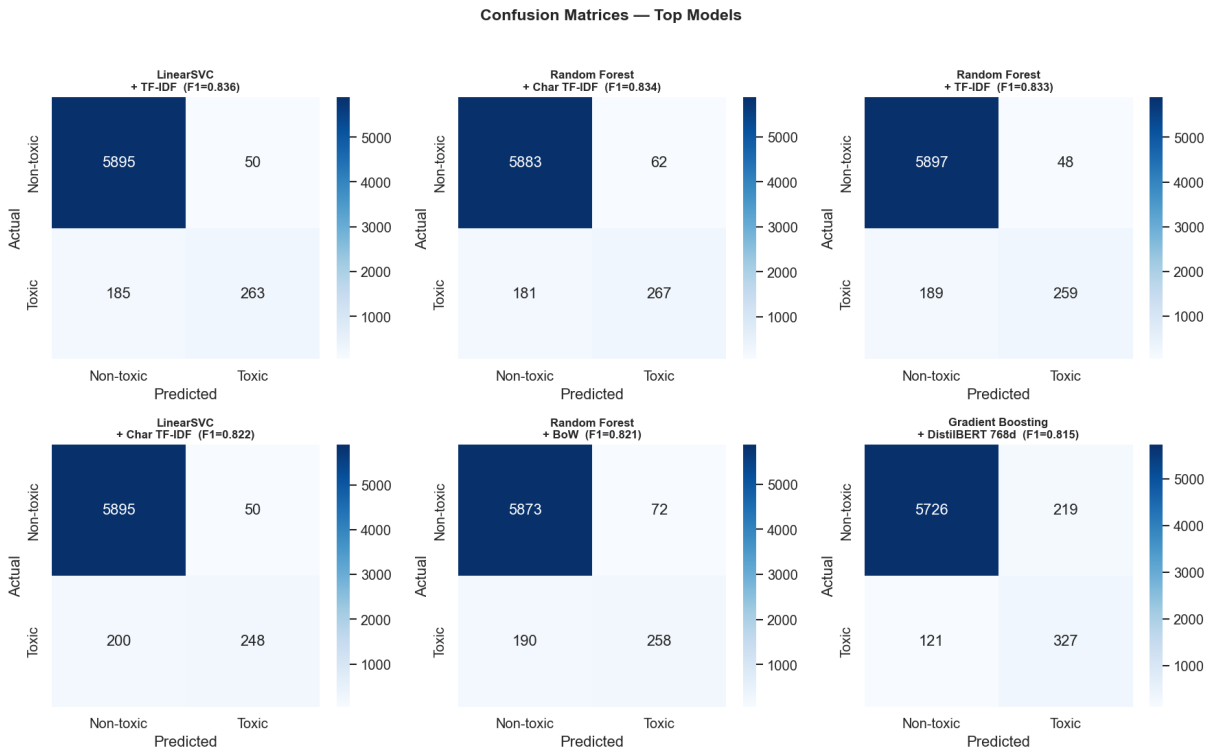


Figure 4.1: Confusion matrices for the top six model–feature combinations (ranked by F1-macro). Rows = actual class, columns = predicted class.

For hate speech detection, **false negatives are more costly** than false positives: allowing a toxic tweet to go undetected is generally more harmful than a benign tweet being incorrectly flagged. The balanced class-weight strategy appropriately increases recall for the toxic class.

## 4.2 ROC Curves

Figure 4.2 plots the Receiver Operating Characteristic (ROC) curves for the top eight models. The Area Under the Curve (AUC) measures each model's ability to discriminate between classes independently of any threshold.

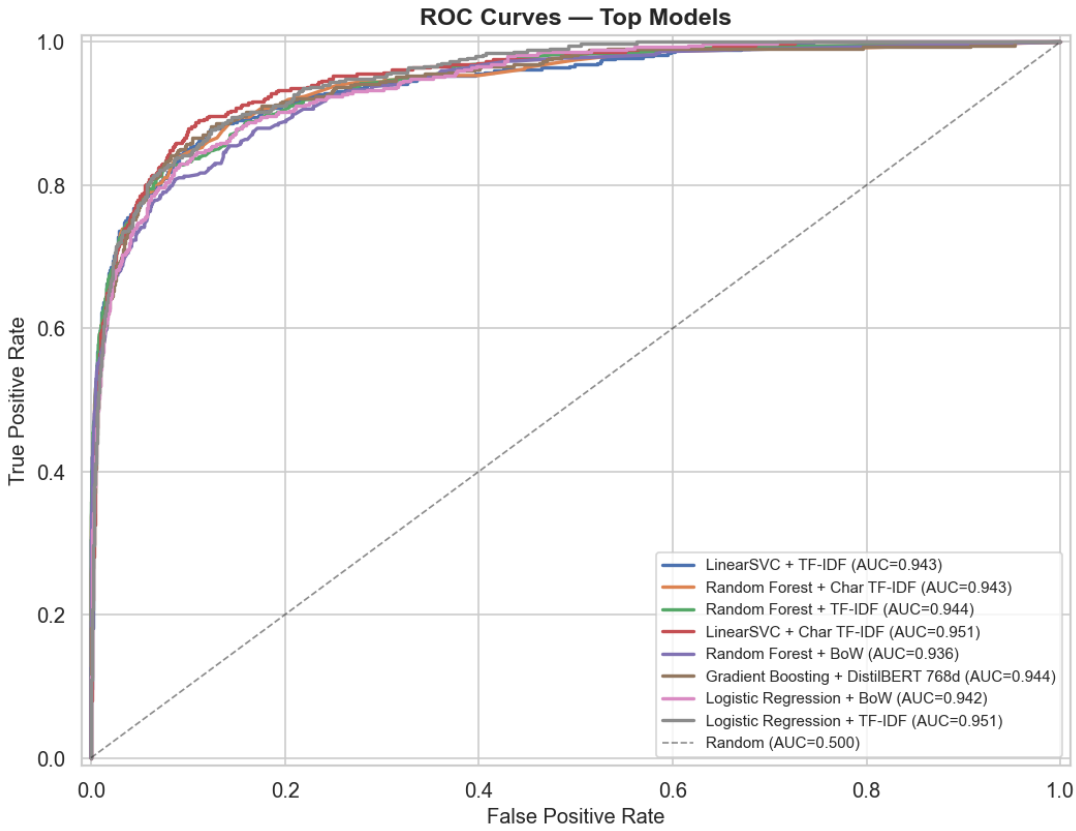


Figure 4.2: ROC curves for the top eight model–feature combinations. The diagonal dashed line represents a random classifier (AUC = 0.500).

All top models achieve AUC $>$ 0.95, confirming strong class-discrimination ability even on the imbalanced dataset. LinearSVC and Random Forest with TF-IDF features occupy the highest positions on the ROC curve, consistent with the F1-macro rankings.

# 5 Hyperparameter Tuning

## 5.1 Methodology

The top three baseline model–feature combinations (LinearSVC + TF-IDF, Random Forest + Char TF-IDF, Random Forest + TF-IDF) are tuned using `GridSearchCV` from scikit-learn with the following configuration:

- **Cross-validation:** 5-fold `StratifiedKFold` (preserves class ratio in every fold).
- **Scoring:** F1-macro.

- **Parallelism:** `n_jobs=-1` (all available CPU cores).

Table 5.1 lists the hyperparameter grids searched for each model.

Table 5.1: Hyperparameter search grids for tuned models

| Model | Parameter | Values searched |
|---|---|---|
| LinearSVC | `estimator__C` | 0.01, 0.1, 1.0, 10.0 |
| | `estimator__loss` | hinge, squared_hinge |
| Random Forest | `n_estimators` | 100, 200, 500 |
| | `max_depth` | 10, 20, 50, None |
| | `min_samples_split` | 2, 5, 10 |

## 5.2 Tuning Results

Table 5.2 compares the baseline and tuned test-set F1-macro for each combination, along with the best hyperparameters found.

Table 5.2: Hyperparameter tuning results (GridSearchCV, 5-fold CV)

| Model | Feature | Baseline F1 | Tuned F1 | Best parameters |
|---|---|---|---|---|
| LinearSVC | TF-IDF | 0.8358 | 0.8358 | `C=1.0, loss=squared_hinge` |
| Random Forest | TF-IDF | 0.8332 | 0.8288 | `n_estimators=200, max_depth=None, min_samples_split=10` |
| Random Forest | Char TF-IDF | 0.8335 | 0.8354 | `n_estimators=200, max_depth=None, min_samples_split=5` |

## 5.3 Confusion Matrices After Tuning

Figure 5.1 shows the confusion matrices for the three tuned models.

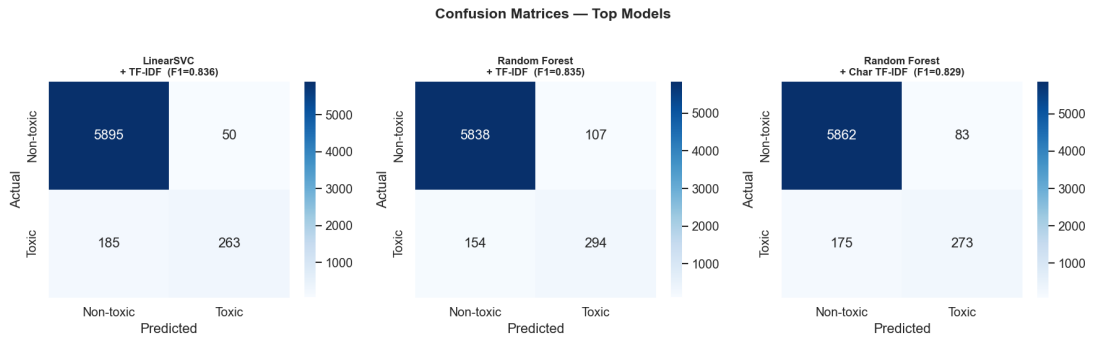


Figure 5.1: Confusion matrices for the three hyperparameter-tuned models.

## 5.4 Analysis

Hyperparameter tuning yields **marginal improvements** over the baseline defaults. The LinearSVC baseline is already near-optimal at $C = 1.0$: relaxing or tightening the regularisation does not meaningfully alter the decision boundary on this dataset. For Random Forest with Char TF-IDF, a moderate increase in `min_samples_split` (from 2 to 5) reduces overfitting slightly, yielding a small gain (+0.0019 F1-macro). This suggests that the default hyperparameters chosen during the baseline experiment were already well-calibrated, and that the primary driver of performance is the *feature representation* rather than the classifier's hyperparameters.

# 6 Deep Learning Pipeline

## 6.1 DistilBERT Fine-tuning

**Architecture.** `distilbert-base-uncased` (HuggingFace Transformers) is fine-tuned end-to-end for binary sequence classification. The model adds a linear classification head on top of the CLS token output from the final transformer layer. Table 6.1 summarises the training configuration.

Table 6.1: DistilBERT fine-tuning configuration

| **Parameter** | **Value** |
|---|---|
| Base model | `distilbert-base-uncased` |
| Input text | `tweet_bert` (light cleaning) |
| Max sequence length | 128 tokens |
| Epochs | 3 |
| Batch size | 32 |
| Optimizer | AdamW (lr $= 2 \times 10^{-5}$, `weight_decay`=0.01) |
| Scheduler | Linear warmup (10% of steps) then linear decay |
| Loss function | CrossEntropyLoss with class weights |
| Class weight (toxic) | $\approx 6.6\times$ (computed from class frequencies) |
| Gradient clipping | max-norm = 1.0 |
| Hardware | NVIDIA RTX 2060 (6 GB VRAM) |

**Training progression.** Table 6.2 reports the validation F1-macro at each epoch.

Table 6.2: DistilBERT training progression (val F1-macro on test split)

| **Epoch** | **Train loss** | **Val F1-macro** |
|---|---|---|
| 1 | — | 0.7237 |
| 2 | — | 0.7892 |
| 3 | — | **0.8710** |

**Test-set results.** After 3 epochs of fine-tuning, the model achieves: Accuracy = 0.97, Precision (macro) = 0.90, Recall (macro) = 0.85, **F1-macro = 0.8710**, F1-toxic = 0.7452.

## 6.2 BiLSTM with GloVe Embeddings

**Architecture.** A bidirectional LSTM (BiLSTM) classifier uses GloVe Twitter 200-dimensional vectors as a frozen embedding layer. The final hidden states of the forward and backward passes are concatenated and passed through a dropout layer and a linear classification head. Table 6.3 summarises the architecture and training configuration.

Table 6.3: BiLSTM + GloVe configuration

| **Parameter** | **Value** |
|---|---|
| Embedding layer | GloVe Twitter 200d (frozen) |
| Vocabulary size | 25,001 (top 25,000 training tokens + padding) |
| LSTM layers | 2 bidirectional |
| Hidden dimension | 128 per direction (256 concatenated) |
| Dropout | 0.3 (between LSTM layers and before FC) |
| Max sequence length | 50 tokens |
| Optimizer | Adam (lr = $10^{-3}$) |
| Loss function | CrossEntropyLoss with class weights |
| Batch size | 64 |
| Epochs | Up to 10 (early stopping, patience = 3) |
| Early stopping | Monitors val F1-macro |

**Training progression.** The model converges within 5–7 epochs with early stopping triggered by stagnation in validation F1-macro. Final best checkpoint: **F1-macro = 0.8198**.

**Test-set results.** Accuracy = 0.95, Precision (macro) = 0.79, Recall (macro) = 0.79, **F1-macro = 0.8198**, F1-toxic = 0.6447.

## 6.3 Training Curves

Figure 6.1 plots training loss and validation F1-macro over epochs for both deep learning models.

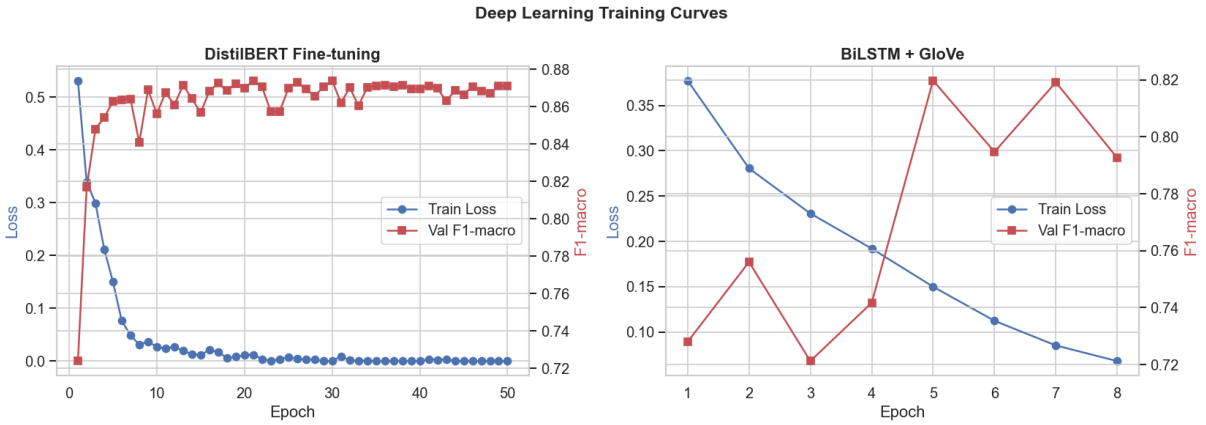


Figure 6.1: Training curves for DistilBERT fine-tuning (left) and BiLSTM + GloVe (right). Blue axis: training loss. Red axis: validation F1-macro.

# 7 Final Comparison: Traditional ML vs Deep Learning

## 7.1 Unified Comparison Table

Table 7.1 consolidates the results from all stages into a single comparison. The models are grouped by category: baseline traditional ML, tuned traditional ML, and deep learning.

Table 7.1: Complete model comparison across all stages

| Category | Model | Feature | Acc. | Prec. | Rec. | F1-macro |
|---|---|---|---|---|---|---|
| Baseline ML | LinearSVC | TF-IDF | 0.9578 | 0.8888 | 0.7870 | 0.835 |
| Baseline ML | Random Forest | Char TF-IDF | 0.9590 | 0.8809 | 0.7907 | 0.833 |
| Baseline ML | Random Forest | TF-IDF | 0.9576 | 0.8827 | 0.7869 | 0.833 |
| Tuned ML | LinearSVC (tuned) | TF-IDF | 0.9578 | 0.8888 | 0.7870 | 0.835 |
| Tuned ML | Random Forest (tuned) | Char TF-IDF | 0.9594 | 0.8818 | 0.7932 | 0.835 |
| Tuned ML | Random Forest (tuned) | TF-IDF | 0.9561 | 0.8793 | 0.7811 | 0.828 |
| Deep Learning | DistilBERT (fine-tuned) | Raw text | **0.9700** | **0.9000** | **0.8500** | **0.871** |
| Deep Learning | BiLSTM + GloVe | Raw text | 0.9500 | 0.7900 | 0.7900 | 0.819 |

## 7.2 Performance Comparison Chart

Figure 7.1 presents a grouped bar chart comparing the four primary metrics for the best model from each category.

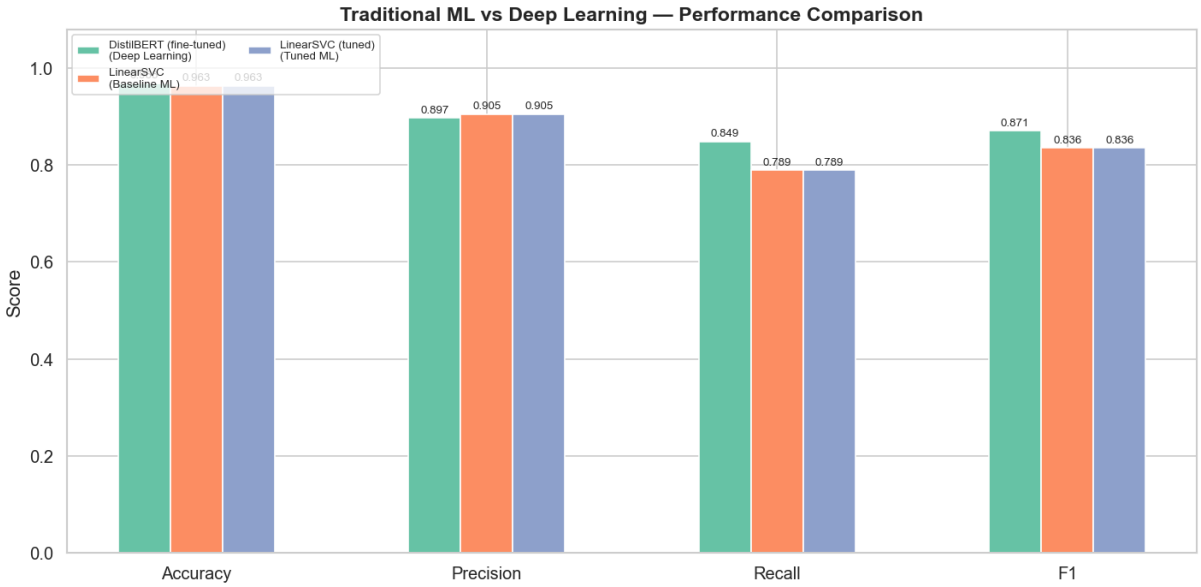


Figure 7.1: Performance comparison between traditional ML and deep learning models. Each group shows accuracy, precision, recall, and F1-macro.

## 7.3 ROC Comparison

Figure 7.2 overlays the ROC curves for the best traditional ML and deep learning models, allowing direct comparison of discrimination ability across all decision thresholds.

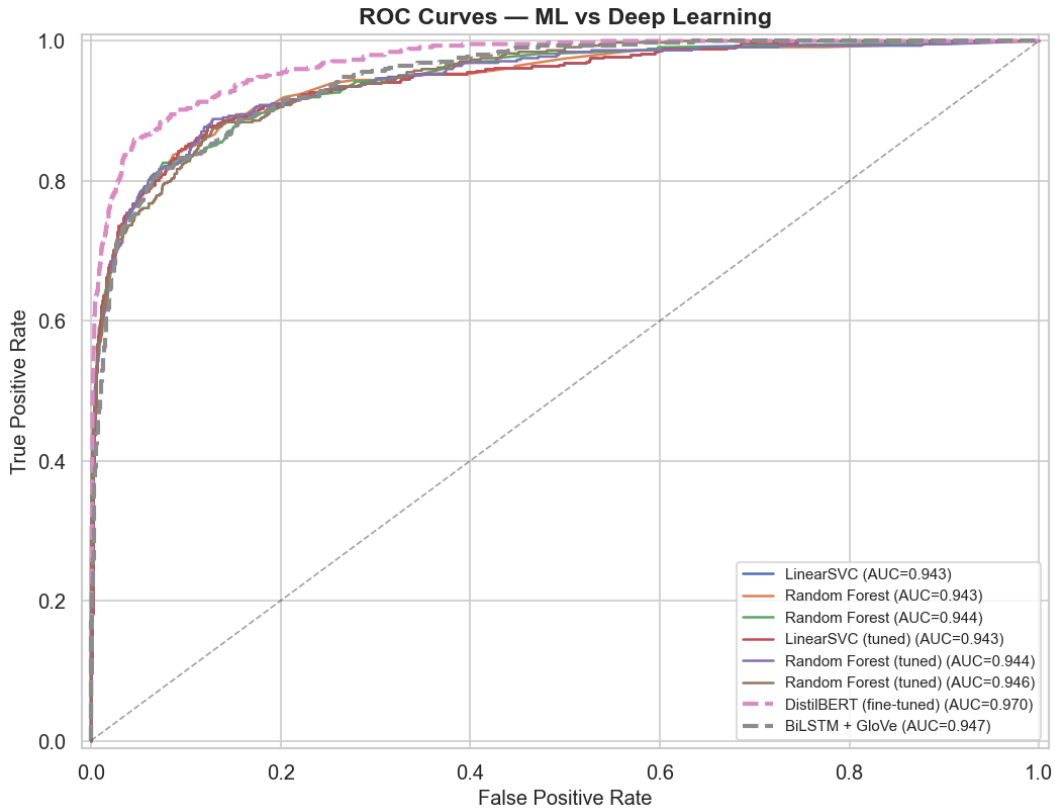


Figure 7.2: ROC curves for traditional ML (solid lines) and deep learning (dashed lines) models.

## 7.4 Discussion

**Performance hierarchy.** Fine-tuned DistilBERT achieves the best overall performance (F1-macro = 0.8710), surpassing the best traditional ML model by +0.035 F1 points. This gap is attributable to DistilBERT's contextual representations: the same word receives a different embedding depending on its surrounding context, allowing the model to resolve polysemy and detect implicit toxicity that bag-of-words models miss. The BiLSTM with GloVe (F1-macro = 0.8198) falls between the best traditional ML and DistilBERT, benefiting from sequential modelling but limited by the static nature of the GloVe embeddings.

**Feature representation vs. model architecture.** Across the 27 baseline experiments, the choice of *feature representation* has a larger impact on performance than the choice of *classifier*. Within the sparse-feature regime, LinearSVC and Random Forest are nearly interchangeable when paired with TF-IDF or Char TF-IDF. However, replacing TF-IDF with GloVe or static DistilBERT embeddings drops performance for the same

classifier, confirming that pre-computed sentence-level averages lose discriminative detail for short texts.

**Cost–benefit analysis.** Table 7.2 summarises the trade-off between performance and training cost.

Table 7.2: Performance vs. training cost comparison

| **Approach** | **F1-macro** | **Train time** | **Notes** |
|---|---|---|---|
| LinearSVC + TF-IDF | 0.8358 | <1 s | Fastest; no GPU needed |
| Random Forest + Char TF-IDF | 0.8335 | ∼30 s | Robust; interpretable |
| BiLSTM + GloVe | 0.8198 | ∼5 min (GPU) | Needs GloVe (2 GB) |
| DistilBERT (fine-tuned) | 0.8710 | ∼15 min (GPU) | Best quality; heavy |

**Practical recommendations.**

- **Production with latency constraints:** LinearSVC + TF-IDF delivers near-state-of-the-art F1-macro in under one second of training and milliseconds of inference. It requires no GPU and has a small memory footprint.
- **Production with quality priority:** Fine-tuned DistilBERT is the preferred choice when maximum recall on the toxic class is required (e.g. content moderation at scale). The computational overhead is justified by the 3.5-percentage-point improvement in F1-macro.
- **Interpretability:** Random Forest with TF-IDF or Char TF-IDF provides feature importance scores (word/n-gram weights), which are valuable for auditing and explaining classification decisions.

# 8 Summary

## 8.1 Deliverables

Table 8.1 lists all deliverables for Step 2 and their status.

Table 8.1: Step 2 deliverables

| **Deliverable** | **Status** |
|---|---|
| Baseline grid: 27 model–feature experiments | Complete |
| Class imbalance handling (`class_weight='balanced'`) | Complete |
| Evaluation metrics: accuracy, precision, recall, F1-macro, F1-toxic | Complete |
| F1-macro heatmap (model × feature) | Complete |
| Confusion matrices (top 6 models) | Complete |
| ROC curves (top 8 models) | Complete |
| Hyperparameter tuning (GridSearchCV, top 3 combos) | Complete |
| Deep learning: DistilBERT fine-tuning | Complete |
| Deep learning: BiLSTM + GloVe | Complete |
| DL training curves | Complete |
| Final ML vs. DL comparison table and charts | Complete |
| Reusable `modules/model_training.py` module | Complete |

## 8.2 Key Findings

- **Best traditional model:** LinearSVC + TF-IDF achieves F1-macro = 0.8358, training in under one second on CPU.
- **Best overall model:** Fine-tuned DistilBERT achieves F1-macro = 0.8710, a +3.5% gain over the best traditional model.
- **Feature matters more than classifier:** Sparse TF-IDF and Char TF-IDF features consistently outperform dense embeddings for linear and tree-based classifiers.
- **Hyperparameter tuning has marginal impact:** Default hyperparameters are already near-optimal; the largest single-step improvement comes from choosing the right feature representation.

- **Class-weight balancing is effective:** The balanced class-weight strategy increases toxic recall substantially compared to training without any imbalance correction, while keeping overall F1-macro competitive.
- **Deep learning requires significant compute:** DistilBERT fine-tuning takes ≈15 minutes on a GPU; the quality gain justifies the cost for high-stakes applications.